
Mastering System Design

Section 7: Database and Storage

Storage

Scalability - Section Agenda

1. Introduction to Storage in System Design
2. Understanding Database Models: SQL vs. NoSQL
3. Advanced Database Topics: Sharding, Replication & Polyglot Persistence
4. Object Storage in Modern Systems
5. File Systems and Distributed Storage
6. Big Data Fundamentals
7. Choosing the Right Storage Solution
8. Summary & Recap

Here is the formatted text:

Hello and welcome to Section 7 of this course on Mastering System Design.

This section is all about databases and storage. In this section, we are diving deep into how systems store,

retrieve and manage data, one of the most critical aspects of any large-scale architecture.

So here is what we will be covering in this section. We will start with the big picture, why storage matters and how it fits into the overall system architecture. Then we will compare the relational database and various NoSQL models like Key-Value, Document, Column and Graph. Then we go deep into advanced database topic. We will explore sharding, replication, cap theorem and polyglot persistence. These are the strategies used to scale database effectively. Then we will learn how systems use services like Amazon S3 to handle large-scale unstructured data. Then we will go under the hood of how data is stored in the traditional and distributed file systems. Then we will look at some of the big data fundamentals. We will touch on how large-scale data is processed and stored, introducing data lakes, batch versus stream processing. Then we will tie everything together with practical guidance on how to evaluate and choose the right storage option for different use cases.

So let's get started with the first lecture on introduction to storage in system design. We are kicking things off with this foundational lecture because we want to cover the basics first and then go into details. So we all know storage is at the heart of almost every system, whether it's a messaging app, a blanking platform or a content delivery network. The way we store, retrieve and manage data has a huge impact on system performance, reliability and scalability.

In this lecture, we will explore the different types of storage. We will dive into the structured versus unstructured data. We will discuss the core storage properties like durability, consistency, etc. And we will see how these decisions shape real-world system design. By the end of this lecture, you will have a strong understanding of why storage isn't just about saving data. It is about architecting for scale, fault tolerance and architecting for future.

Why Storage Matters in System Design

- All systems generate and consume data — storing it is essential
- Storage impacts performance, reliability, cost
- Persistent storage enables everything from user profiles to search history to analytics



So let's start with a very simple truth. Every system generates and consumes data, whether it's a user input, logs, media file or analytics. But it's not enough to generate data. We need a way to store it reliably, retrieve it efficiently and manage it at scale. That's where storage becomes foundational to system design.

Persistent storage is what powers features like user profiles, save preferences, watch history, shopping carts and even things like recommendations or analytics dashboard.

And it's not just about storing data. Your choice of storage system directly impacts system performance, reliability and cost. For example, slow storage can bottleneck your response time and unreliable storage can result in data loss or downtime. That's why thinking about storage early in the design process is very critical. It should not be an afterthought, but it should be done early when we are building scalable and fault tolerant systems.

Structured vs. Unstructured Data

- **Structured:** Rows/columns, predefined schema (e.g., SQL tables)
- **Unstructured:** No schema, flexible format (e.g., images, videos, logs)

STRUCTURED DATA FORMATS



UNSTRUCTURED DATA FORMATS



So to design effective storage solutions, it's important to understand the nature of the data we are working with. And that starts with the distinction between structured and unstructured data.

Structured data is highly organized and stored in a well-defined format, typically rows and columns. Think of a SQL database table where fields like user ID, email, created at etc. exist. The schema is predefined, which makes structured data easy to query using languages like SQL.

Unstructured data, on the other hand, doesn't follow a rigid schema. It's flexible in format and often comes as images, videos, PDFs, log files or social media posts. You can't just run a simple SQL to get the data. Instead, the data is stored in object stores or blob storage and analyzed differently.

Both types are common in modern systems. For example, an e-commerce app might store user information as structured data and product images or reviews as unstructured data.

Understanding this distinction helps us choose the right storage technology and design for performance, scalability and retrieval efficiency.

Categories of Storage

- Database storage (SQL/NoSQL)
- Object storage (e.g., S3)
- File storage (e.g., NFS)
- Block storage (e.g., EBS)



Here is the formatted text:

So now that we understand the type of data, let's explore the different categories of storage available in system design. Each one is optimized for a different use case.

First, we have database storage. This is your traditional SQL or NoSQL database used to store structured or semi-structured data. SQL databases like PostgreSQL are great for strong consistency and complex queries, while NoSQL options like MongoDB or Cassandra offer flexibility and horizontal scalability.

Next is object storage. Like Amazon S3 or Google Cloud Storage, this is ideal for unstructured data such as images, videos, logs or backup. It is built for durability and scalability and objects are accessed via APIs.

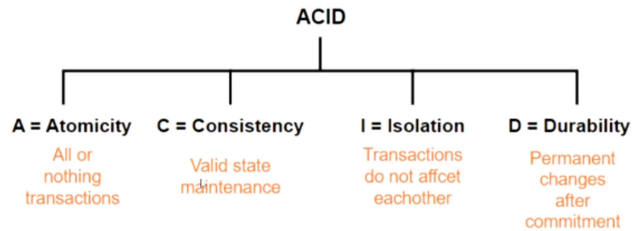
Then there is file storage, which provides a hierarchical file system like NFS or SMB. It is commonly used in legacy systems or environments that need shared access to file over the network.

And then finally, we have block storage. These block storages provide low latency, high performance access to raw storage blocks. It is perfect for databases or virtual machines that need direct disk access and fast IO.

So choosing the right storage category depends on the data type, access pattern, performance needs and scale of our application.

Storage Properties

- Durability – Data persists even after failures
- Availability – Data can be accessed when needed
- Consistency – Every read returns the most recent write
- (Optional) Atomicity – Operations are all-or-nothing (relevant in transactional storage)



So when choosing or designing a storage system, we need to understand the core properties that define its behavior, especially in distributed environments.

So what are these core properties?

The first is durability. This means that once data is written, it will persist even in case of power loss, crashes or hardware failures. A durable system ensures that your data won't just disappear.

Second is availability. Availability refers to the system's ability to serve data when it is requested, even during partial outages or node failures. High availability is critical for systems that can't afford downtime like payment platforms or messaging apps.

Then comes consistency. Consistency ensures that every read reflects the most recent write. In simpler terms, once you update something, say user profile or any amount in your account table, anyone fetching that data sees the latest version.

And then finally, and optionally, we also should talk about atomicity, which is especially relevant in transactional systems. It means operations are either all or nothing. Either everything succeeds or nothing succeeds. This is vital when you are updating multiple records and systems need to stay in a valid state even after the update.

So these properties, durability, availability, consistency and atomicity, forms the foundation of evaluation when it comes to finding trade-offs and selecting the right storage mechanism for our application.

The Trade-offs in Storage Design

Scalability vs. Reliability vs. Performance

- No perfect solution — must trade off between:
 - Scale (can handle growth)
 - Reliability (resistant to failure)
 - Performance (speed of reads/writes)



So in the real world, storage design is all about trade-offs because unfortunately there is no silver bullet that gives you everything. So you often have to balance three key factors, scalability, reliability and performance.

Scalability means that the system can handle increased load, more data, more user, more request without breaking down. It is essential for growing systems.

Reliability is all about ensuring that the system continues to function even when parts of it fails. It could be done

either through replication, failover mechanism and strong data guarantees.

Performance refers to the fast system access, how fast system can read and write data, which directly impacts user experience and back-end responsiveness.

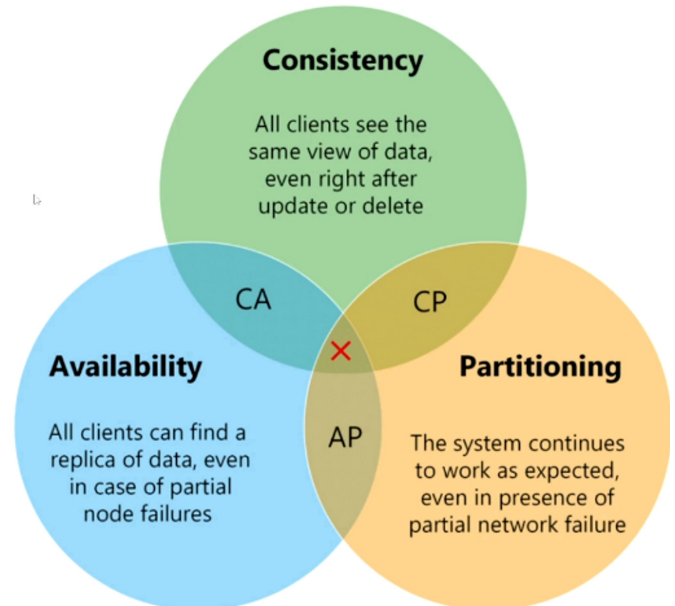
But there is a challenge. Optimizing for one of these usually comes at the expense of the other. A super performance system might not scale well. A highly reliable system might involve latency due to replication overheads.

So what to do? Well, every architectural decision comes down to what your system needs most and where you are willing to compromise. So this is where system designers add real value by making smart, context-driven trade-offs that align with business goals and technical realities.

The CAP Theorem

- In any distributed system, you can only fully guarantee 2 of the 3:
 - Consistency – Every read gets the latest write
 - Availability – Every request receives a response
 - Partition Tolerance – System continues despite network failures
 - No system can have all 3 at the same time

Note: Since network partitions are unavoidable, especially at scale, real-world systems must choose between Consistency and Availability during a partition.



Here is the formatted text:

Now one of the most important concepts in distributed systems is the CAP theorem and it directly impacts how we design and choose our storage system.

So CAP stands for Consistency, Availability and Partition Tolerance. This theorem states that in any distributed system, you can only fully guarantee two out of three at any given point of time.

So let's try to break it down. Consistency means every read returns the most recent write. No stale data is coming back. Then availability means every request gets a response, even if it's not most up-to-date. And then finally, the most confusing one, Partition Tolerance means that the system continues to operate even if there is a network failure or some nodes can't communicate. What it means is if your database is partitioned and it exists at multiple nodes, your system should still be operational if some of the nodes are not available. It could be due to network failure or the node's failure itself.

Now, what is the catch? According to CAP theorem, you can't have all three at once in the presence of a network partition, which is almost inevitable in the large scale systems. So you are forced to choose between consistency and availability.

So different systems make different tradeoffs. Some prioritize consistency and partition tolerance, others choose availability and partition tolerance. And the systems that offer consistency and availability, but does not tolerate partition are very rare, but are not really distributed.

Understanding this CAP theorem helps us make smart architectural choices based on the system's tolerance for stale data, downtime and failures.

Types of Systems Based on CAP Trade-offs

- CP (Consistency + Partition Tolerance)
 - Prioritizes data correctness over availability
 - During a partition, the system may reject requests to avoid inconsistent reads
 - Not always available, but when it is — data is guaranteed to be correct
 - Example: HBase: Strongly consistent. If a node can't confirm a write across replicas, it won't serve it — even if it means being unavailable briefly.
 - When to use: Financial systems, banking apps, anything where data integrity is critical
- AP (Availability + Partition Tolerance)
 - Prioritizes system uptime over consistency
 - During a partition, the system will serve requests, even if they return stale or eventually consistent data
 - Example: DynamoDB: Inspired by Amazon's Dynamo model, which uses eventual consistency by default for high availability
 - When to use: Social media feeds, product catalogs, content delivery — where being up is more important than perfect accuracy
- CA (Consistency + Availability) — The “Unicorn”
 - Only possible if no network partitions ever occur — i.e., in single-node or tightly coupled systems
 - In practice, not achievable in distributed systems that need to tolerate network faults
 - Example: Relational databases (like PostgreSQL) in standalone mode (not distributed) could be considered CA